

1. Huffman Tree Using PQ**12A1**

我们的示例代码分别基于三种数据结构——**List**、**ComplHeap**和**LeftHeap**——实现了Huffman算法。

- 在掌握了**ComplHeap**和**LeftHeap**之后，试找到对应的项目，分别编译、运行和测试。
- 确认这三个项目均采用了**统一**的算法框架，也就是说，对应的源代码实际上是同一个文件。
- 三种实现方式在**各方面**的性能有何差异？

2. AVL As PQ**12A2**

- 试基于**AVL**树，通过派生来实现**PQ**。
- 这种实现方式的**空间**复杂度是多少？各接口的**时间**复杂度又如何？

3. Vector/List As PQ**12A2**

本节简略介绍了基于无序、有序的向量、列表实现PQ的方式，并列出了主要接口对应的效率。

如果只限于这四种实现方式，在**实际应用**中你会更加倾向于哪种方式？为什么？

4. Multiple Inheritance**12B1**

学习或温习C++的**多重继承**特性，理解这里如何利用这一特性，由PQ和Vector派生出ComplHeap。

5. insert() & percolateUp()**12B2**

课上已简要说明，`PQ_ComplHeap::insert()`操作**期望**地只需**常数**时间。试**尽可能**精确地估算出这个常数。

6. percolateUp()**12B2**

所谓的**上滤**过程，无非是一系列的**交换**（swap）操作——每次交换都需通过三次**移动**（move）操作来完成——最坏情况下累计约 $3 * \log n$ 次。然而实际上不难看出，在整个的**上滤**过程中，参与各次**交换**的总有一个元素是**固定**的——即被插入/上滤的那个元素。试基于这个事实优化示例代码，使得即便是在最坏情况下，每次下滤累计也只需做约 $1 * \log n$ 次**移动**操作。

7. remove() & percolateDown()**12B3**

试证明，`PQ_ComplHeap::delMax()`操作**期望**地仍然需要 $O(\log n)$ 时间。

8. percolateDown()**12B3**

试参照前面对**插入/上滤**算法的改进方法，在完全二叉堆的**删除/下滤**过程中减少所需执行的**移动**操作。

9. heapify()**12B4**

Floyd算法可以保证在 $O(n)$ 时间内，将 n 个随机元素**构建**为一个完全二叉堆。但由以上分析可知，就**期望**复杂度而言，直接调用 n 次`insert()`的朴素方法也是 $O(n)$ 。两种方法的**实际**效率，孰优孰劣？

10. heapify()**12B4**

- 试验证，Floyd算法中for循环的条件判断“`-1 != i`”可以**等效**地改作“`i < n`”。
- 为什么我们还是更倾向于**前一种**实现方式？

11. Cache**12B**

我们知道，ComplHeap在底层实质上是借助一个**向量**来存放所有元素的。

那么，这是否意味着，系统**缓存**因此可以得到**充分利用**？为什么？

12. Kruskal Using ComplHeap**12B**

我们知道，Kruskal算法需要**从短到长**地逐条尝试图中各边，但首先就用 $\mathcal{O}(e \log e) = \mathcal{O}(e \log n)$ 的“**投资**”将所有的边按长度**排序**，往往失之鲁莽。若改用**小顶**的ComplHeap仅维护一个“**廉价**”的**偏序**，则这方面的时间成本（借助Floyd算法）可以降至 $\mathcal{O}(n)$ ——尽管此后每次还需另花费 $\mathcal{O}(\log n)$ 时间取出下一条边。

- 试分别按两种方式实现Kruskal算法。
- 两个算法在时间性能方面的相对优劣，**主要**取决于哪个（些）因素？

13. Double-Ended Priority Queue**12B**

- 所谓的**双端优先级队列**，在insert()之外，还同时兼顾delMax()和delMin()接口。

该ADT最原始的一种实现方式，无非是同时维护“**对称的**”两个堆：一个大顶堆，另一个小顶。

这种实现方式的时间、空间复杂度是多少？主要的缺点在哪里？

- DEPQ有多种更为高效的实现方式，比如**对称最小最大堆**（SMMH, Symmetric Min-Max Heap），仅使用一个树形结构，每个元素在其中只有一份记录。试搜索并查阅相应的材料，理解其原理及算法过程。
- 在示例代码包中找到对应的项目，阅读代码，并编译、运行和测试。

14. Heapsort**12B5**

如果**调转**方向，将待/已排序的元素放在前/后缀，在实现堆排序算法时会有什么问题？

15. Heapsort**12B5**

为便于学习理解，讲义中的堆排序算法过于追求**形式**上的简洁。

试改写该算法，以（在常系数意义上）减少元素**移动**的次数。

16. Replay In A Winner Tree**12C1**

试具体地实现**胜者树**结构：说明各组成部分及其关系，并针对**重赛**算法做一解释和分析。

17. Winner Tree vs. ComplHeap**12C1**

- 为了从10,000个整数中选出最大的5个，试分别基于**胜者树**、**完全二叉堆**各设计并实现一个算法。
- 试从**空间**消耗量、**时间**成本的角度，对两个算法做一分析对比。

18. Replay In A Loser Tree**12C2**

试具体地实现**败者树**结构：说明各组成部分及其关系，并针对**重赛**算法做一解释和分析。

19. Loser Tree ~ 1D MLST**12C2 + 07C3**

实际上从某个角度来看，07D1节介绍过的**1维MLST**结构，也可以视作为本节介绍的**败者树**。

那么在这棵**败者树**中，各节点的**优先级数**是什么？**优先级关系**是如何约定和判断的？

20. d-Ary Heap**12D**

采用多叉堆来优化PFS算法的理论依据是，下滤操作通常都**远少于**上滤。具体地，本节分别用顶点数 n 、边数 e 来估计这两种操作的次数。然而实际上，尽管**前**一估计是**准确的**，但**后**一估计只是一个**上界**：**并非**每次调用prioUpdater()都会引发上滤；即便引发上滤，节点上升的高度也**未必**达到 $\mathcal{O}(\log n)$ 。

- 鉴于以上原因，本节针对分叉数所给的**建议值**（ $d = e/n + 2$ ）相对于**最优值**是**偏大**还是**偏小**了？
- 你觉得可以从哪些方面着手，**更加**精准地做出估计？

21. Multiple Inheritance**12E1**

学习或温习C++的**多重继承**特性，理解这里如何利用这一特性，由PQ和BinTree派生出LeftHeap。

22. LeftHeap::merge()**12E3**

- a) **递归版**、**迭代版**中的短路求值和特判，分别针对什么特殊情况？试分别给出具体的实例。
- b) **迭代版**首先通过**特判**，处理了堆a或/和b为空的情况。

如果只是用merge()来实现PQ的标准接口insert()和delMax()，这些**特判**是否**必要**？为什么？

- c) **迭代版**的二路归并过程中，堆a、堆b中节点的**归入方式**很不相同，试对照代码分析体会。
- d) 以本节针对**递归版**的实例，对照**迭代版**做一推演。两种合并实现方式的**结果**是否一致？

23. Degeneracy**12E4**

如讲义中的代码所示，左式堆的insert()、delMax()均可高效地转化并实现为merge()操作。

- a) 试进一步确认，即便是在**退化情况**下（比如插入**首元素**、删除**末元素**等），示例代码也是**安全且正确**的。
- b) 试在示例代码包中找到对应的项目，通过**实际测试**验证上述分析结论。
- c) 如此实现的这两个接口，在**最好**情况下分别需要多少时间？

24. More PQ's**12**

试自行查阅资料，自学优先级队列家族中的更多变种，比如**斜堆**、**二项式堆**、**Fibonacci堆**等等。

- a) 这些变种的insert()和delMax()接口，时间复杂度各是多少？

比如，其中的**Fibonacci堆**是对左式堆的改进，其**上滤算法**采用了**懒惰合并**的策略。

- b) 试确认，该结构的insert()、merge()、increase()等接口，**均摊**时间复杂度都是 $O(1)$ ；而delMax()接口的**均摊**时间复杂度，却仍是 $O(\log n)$ 。

试证明，无论如何实现：

- c) PQ::delMax()接口在**最坏**情况下的时间复杂度，都不可能低于 $\Omega(\log n)$ 。
- d) PQ::delMax()接口在**平均**情况下的时间复杂度，都不可能低于 $\Omega(\log n)$ 。